

Лабораторна робота №4

Дослідження дерев рішень та ансамблевих методів
у задачі багатокласової класифікації
за наявності пропущених значень

Мета роботи

Дослідити та порівняти три алгоритми класифікації — дерево рішень (**Decision Tree**), випадковий ліс (**Random Forest**) та градієнтний бустинг (**XGBoost**) — на реальному датасеті **Annealing**, що містить пропущені значення. Основну увагу приділити:

- впливу способу обробки пропусків на якість класифікації;
- коректному (без витоку даних) підбору гіперпараметрів за допомогою **Optuna**;
- неупередженому оцінюванню моделей (вкладена крос-валідація) та статистичному порівнянню результатів;
- інтерпретації моделей і розбіжностей між їхніми прогнозами (permutation importance, SHAP).

Очікувані результати навчання

Після виконання роботи студент повинен уміти:

1. аналізувати структуру пропусків у даних і формулювати гіпотезу щодо механізму їх виникнення (MCAR / MAR / MNAR);
2. будувати пайплайни попередньої обробки в **scikit-learn**, що виключають витік даних;
3. налаштовувати гіперпараметри так, щоб оцінка якості моделі не була оптимістично зміщеною;
4. обґрунтовано порівнювати моделі з урахуванням дисперсії оцінок, а не лише середніх значень;
5. пояснювати як глобальну поведінку моделі, так і окремі прогнози.

Програмне забезпечення

Python ≥ 3.10 , `scikit-learn` ≥ 1.4 , `xgboost` ≥ 2.0 , `optuna` ≥ 3.0 , `shap` ≥ 0.44 , `pandas`, `matplotlib/seaborn`. Версії бібліотек, використаних у роботі, необхідно зафіксувати у звіті (наприклад, `pip freeze > requirements.txt`).

Індивідуальний варіант

Усі джерела випадковості (поділ даних, крос-валідація, моделі, семплер `Optuna`) ініціалізуються однією константою:

$$\text{SEED} = 42 + N,$$

де N — номер студента у списку групи. Через це числові результати в кожного студента будуть різними, а висновки мають ґрунтуватися на власних результатах.

1 Короткі теоретичні відомості

1.1 Механізми виникнення пропусків

За класифікацією Рубіна [5] розрізняють три механізми:

- **MCAR** (Missing Completely At Random) — імовірність пропуску не залежить ні від спостережуваних, ні від неспостережуваних даних;
- **MAR** (Missing At Random) — імовірність пропуску залежить лише від спостережуваних ознак;
- **MNAR** (Missing Not At Random) — імовірність пропуску залежить від самого відсутнього значення або від неспостережуваних чинників.

Якщо пропуски не є MCAR, сам факт пропуску може нести інформацію про цільову змінну. Саме тому бінарні індикатори пропусків (варіант В нижче) здатні покращити якість прогнозу [7]. В окремих датасетах частина пропусків є *структурними*: ознака просто не застосовна до цього об'єкта, і її “заповнення” медіаною чи модою не має фізичного змісту.

1.2 Оптимістичне зміщення під час підбору гіперпараметрів

Якщо гіперпараметри обрано за найкращим середнім значенням метрики на крос-валідації, то це саме значення є *оптимістично зміщеною* оцінкою якості: ми обрали максимум серед багатьох зашумлених оцінок [8]. Неупереджену оцінку дає **вкладена крос-валідація** (nested CV): зовнішній цикл оцінює якість, внутрішній — підбирає гіперпараметри лише на тренувальній частині кожного зовнішнього блоку.

1.3 Порівняння моделей за результатами крос-валідації

Блоки крос-валідації перетинаються за тренувальними даними, тому оцінки на різних блоках не є незалежними, і звичайний парний t -тест занижує дисперсію. Для

повторної k -блочної крос-валідації (r повторень) використовують **скоригований t -тест Надо–Бенжіо** [9]:

$$t = \frac{\bar{d}}{\sqrt{\left(\frac{1}{kr} + \frac{n_{\text{test}}}{n_{\text{train}}}\right) \hat{\sigma}_d^2}}, \quad \text{ступенів свободи: } kr - 1,$$

де d_j — різниця метрик двох моделей на j -му блоці (обидві моделі оцінюються на *одних і тих самих* блоках), $\bar{d}$ і $\hat{\sigma}_d^2$ — вибіркові середнє та дисперсія d_j , $n_{\text{test}}/n_{\text{train}}$ — відношення розмірів валідаційної й тренувальної частин блоку.

2 Опис датасету

Датасет **Annealing** описує параметри промислового процесу відпалу сталі. Його можна завантажити з OpenML:

```
1 from sklearn.datasets import fetch_openml
2
3 data = fetch_openml(name='anneal', version=1, as_frame=True)
4 X, y = data.data, data.target
```

Основні характеристики:

- кількість спостережень: 898 (OpenML) або 798 (навчальна частина в UCI) — у звіті вказати, яку версію використано;
- 38 ознак: числові, бінарні та категоріальні;
- цільова змінна: багатокласова, із **сильним дисбалансом** — один клас домінує, а найменший містить лише кілька спостережень; один із класів, описаних у документації, у даних відсутній;
- багато ознак мають дуже високу частку пропусків, причому частина з них є структурними (див. документацію датасету `anneal.names`).

Ефект стелі

На цьому датасеті деревовидні моделі зазвичай досягають асигасу, близької до 1. Через це різниці між моделями невеликі й легко можуть бути шумом. Саме тому в роботі важливі масго F1, стандартне відхилення та статистична перевірка відмінностей, а не лише “у кого більше”.

3 Хід роботи

3.1 Первинний аналіз даних

1. Завантажити датасет, визначити матрицю ознак X і цільову змінну y . Привести категоріальні ознаки до типу `object`.
2. Визначити кількість спостережень, ознак та їхні типи.

3. Обчислити частку пропусків для кожної ознаки. Ознаки, що є повністю порожніми або константними, видалити з обґрунтуванням.
4. Проаналізувати розподіл цільової змінної. Для класів із дуже малою кількістю спостережень обґрунтувати рішення (залишити, об'єднати або виключити) та пояснити, як воно вплине на крос-валідацію (див. “Типові помилки”).
5. **Аналіз механізму пропусків.** Для 3–5 ознак із найбільшою часткою пропусків побудувати таблицю спряженості “пропуск / не пропуск” $\times$ клас і перевірити незалежність критерієм χ^2 (або точним критерієм Фішера для малих частот). Зробити висновок: чи схожі пропуски на MCAR, чи вони пов’язані з класом?
6. Закодувати цільову змінну цілими числами $0, \dots, K-1$ за допомогою `LabelEncoder` (цього вимагає `XGBoost`).

3.2 Поділ даних і пайплайни обробки

Виконати стратифікований поділ. Тестова вибірка використовується **один раз** — для фінальної оцінки в п. 3.6.

```
1 X_train, X_test, y_train, y_test = train_test_split(
2     X, y_enc, test_size=0.2, stratify=y_enc, random_state=SEED
3 )
```

Побудувати два варіанти обробки.

Варіант А (базова імпутація): числові ознаки — медіана; категоріальні — мода; one-hot кодування категоріальних ознак.

Варіант В (імпутація з індикаторами пропусків): як варіант А, плюс бінарні ознаки-індикатори для кожної ознаки, що містила пропуски *у тренувальних даних*.

```
1 from sklearn.compose import ColumnTransformer, make_column_selector
2 from sklearn.impute import SimpleImputer, MissingIndicator
3 from sklearn.pipeline import Pipeline
4 from sklearn.preprocessing import OneHotEncoder
5
6 def build_preprocessor(with_indicators: bool) -> ColumnTransformer:
7     transformers = [
8         ('num', SimpleImputer(strategy='median'),
9          make_column_selector(dtype_include='number')),
10        ('cat', Pipeline([
11            ('impute', SimpleImputer(strategy='most_frequent')),
12            ('onehot', OneHotEncoder(handle_unknown='ignore',
13                                     sparse_output=False)),
14        ]), make_column_selector(dtype_exclude='number')),
15    ]
16    if with_indicators:
17        transformers.append(
18            ('miss', MissingIndicator(features='missing-only'),
19             make_column_selector()))
20    return ColumnTransformer(transformers)
21
22 def build_pipeline(name, with_indicators=True, params=None):
23     return Pipeline([('prep', build_preprocessor(with_indicators)),
24                      ('clf', make_model(name, params))])
```

Примітка. Починаючи з `scikit-learn 1.3–1.4`, `DecisionTreeClassifier` і `RandomForestClassifier` уміють працювати з `NaN` нативно, як і `XGBoost`. У цій роботі імпуція застосовується навмисно, щоб усі моделі отримували однакові вхідні дані й порівняння було контрольованим. Нативна обробка пропусків досліджується в додатковому завданні.

3.3 Базові моделі

Для кожного варіанту обробки навчити три моделі з типовими гіперпараметрами:

```
1 from sklearn.tree import DecisionTreeClassifier
2 from sklearn.ensemble import RandomForestClassifier
3 from xgboost import XGBClassifier
4
5 def make_model(name, params=None):
6     params = params or {}
7     if name == 'DT':
8         return DecisionTreeClassifier(random_state=SEED, **params)
9     if name == 'RF':
10        return RandomForestClassifier(random_state=SEED, n_jobs=-1, **
11        params)
12    return XGBClassifier(random_state=SEED, n_jobs=-1,
13                          eval_metric='mlogloss', **params)
```

Оцінювання виконувати на тренувальній вибірці **повторною стратифікованою крос-валідацією** (5 блоків $\times$ 3 повторення) за метриками accuracy, macro F1 (основна) та balanced accuracy:

```
1 from sklearn.model_selection import RepeatedStratifiedKFold,
   cross_validate
2
3 scoring = {'acc': 'accuracy', 'f1m': 'f1_macro',
4           'bacc': 'balanced_accuracy'}
5 cv = RepeatedStratifiedKFold(n_splits=5, n_repeats=3, random_state=SEED)
6 res = cross_validate(build_pipeline('RF', with_indicators=True),
7                      X_train, y_train, cv=cv, scoring=scoring)
```

Усі моделі й варіанти мають оцінюватися на **одному й тому самому** об'єкті `cv` — це потрібно для парного статистичного порівняння в п. 3.5. Результати подати у форматі середнє $\pm$ стандартне відхилення (табл. 1 і 2).

3.4 Підбір гіперпараметрів за допомогою Optuna

Оптимізацію виконувати для варіанту В. Критично важливо, щоб у цільову функцію передавався **весь пайплайн** (обробка + модель), а не заздалегідь оброблені дані: інакше імпутер “бачить” валідаційні блоки і виникає витік даних.

```
1 import optuna
2 from sklearn.model_selection import StratifiedKFold, cross_val_score
3
4 def suggest_params(trial, name):
5     if name == 'DT':
6         return {
7             'max_depth': trial.suggest_int('max_depth', 2, 20),
8             'min_samples_split': trial.suggest_int('min_samples_split',
9             2, 20),
10            'min_samples_leaf': trial.suggest_int('min_samples_leaf', 1,
11            10),
```

```

10         'class_weight': trial.suggest_categorical(
11             'class_weight', [None, 'balanced']),
12     }
13     if name == 'RF':
14         return {
15             'n_estimators': trial.suggest_int('n_estimators', 100, 500,
16 step=50),
17             'max_depth': trial.suggest_int('max_depth', 3, 30),
18             'min_samples_leaf': trial.suggest_int('min_samples_leaf', 1,
19 10),
20             'max_features': trial.suggest_categorical(
21                 'max_features', ['sqrt', 'log2', 0.5]),
22             'class_weight': trial.suggest_categorical(
23                 'class_weight', [None, 'balanced', 'balanced_subsample'
24 ]),
25         }
26     return { # XGB
27         'n_estimators': trial.suggest_int('n_estimators', 50, 400),
28         'max_depth': trial.suggest_int('max_depth', 2, 10),
29         'learning_rate': trial.suggest_float('learning_rate', 0.01, 0.3,
30 log=True),
31         'subsample': trial.suggest_float('subsample', 0.6, 1.0),
32         'colsample_bytree': trial.suggest_float('colsample_bytree', 0.6,
33 1.0),
34         'min_child_weight': trial.suggest_float('min_child_weight', 1,
35 10, log=True),
36     }
37
38 def tune(name, X_tr, y_tr, n_trials=30):
39     inner = StratifiedKFold(n_splits=3, shuffle=True, random_state=SEED)
40     def objective(trial):
41         pipe = build_pipeline(name, True, suggest_params(trial, name))
42         return cross_val_score(pipe, X_tr, y_tr, cv=inner,
43                                 scoring='f1_macro').mean()
44
45     study = optuna.create_study(
46         direction='maximize',
47         sampler=optuna.samplers.TPESampler(seed=SEED))
48     study.optimize(objective, n_trials=n_trials)
49     return study

```

Оцінка якості після оптимізації — вкладена крос-валідація. Для кожної моделі виконати зовнішню 5-блочну стратифіковану крос-валідацію; на кожному зовнішньому блоці запускати `tune()` лише на його тренувальній частині, а якість обчислювати на валідаційній:

```

1 from sklearn.metrics import f1_score
2
3 outer = StratifiedKFold(n_splits=5, shuffle=True, random_state=SEED)
4 nested_scores = []
5 for tr_idx, va_idx in outer.split(X_train, y_train):
6     X_tr, y_tr = X_train.iloc[tr_idx], y_train[tr_idx]
7     X_va, y_va = X_train.iloc[va_idx], y_train[va_idx]
8     study = tune('XGB', X_tr, y_tr)
9     model = build_pipeline('XGB', True, study.best_params).fit(X_tr,
10 y_tr)
11     nested_scores.append(f1_score(y_va, model.predict(X_va), average='
12 macro'))

```

Результати вкладки крос-валідації навести в табл. 3 разом зі значенням `study.best_value`

однієї “звичайної” оптимізації на всій тренувальній вибірці. Порівняти ці дві величини та пояснити різницю.

Фінальні моделі. Запустити `tune()` на всій тренувальній вибірці, навчити кожну модель із найкращими параметрами (табл. 4) і зберегти. Для кожної моделі побудувати графіки `optuna.visualization.matplotlib.plot_optimization_history` та `plot_param_importances` і коротко прокоментувати: чи збіглася оптимізація, які параметри найважливіші?

3.5 Статистичне порівняння

На основі результатів п. 3.3 (однакові блоки!) скоригованим *t*-тестом Надо–Бенжіо (розд. 1.3) перевірити при $\alpha = 0,05$:

1. чи значуща різниця масго F1 між варіантами A і B для кожної з трьох моделей;
2. чи значуща різниця між двома найкращими моделями у варіанті B.

Реалізувати тест самостійно (10–15 рядків коду, *p*-значення — через `scipy.stats.t`). Результати подати в табл. 5: $\bar{d}$, *t*, *p*, висновок.

3.6 Фінальна оцінка на тестовій вибірці

Для трьох фінальних моделей (варіант B, після оптимізації) обчислити ассурау, масго F1, balanced ассурау на тестовій вибірці (табл. 6), а також `classification_report` для найкращої моделі. Пояснити, чому тестові значення можуть відрізнятися від результатів крос-валідації (зокрема з огляду на кількість об’єктів найменших класів у тесті).

Для найкращої моделі побудувати абсолютну та нормалізовану (по рядках) матриці плутанини. Визначити, які класи найчастіше плутаються між собою.

3.7 Інтерпретація: важливість ознак

Для трьох фінальних моделей обчислити два види важливості ознак і побудувати графіки top-15:

1. **MDI** (`feature_importances_`) — важливість на основі зменшення неоднорідності; обчислюється на *оброблених* ознаках (після one-hot та з індикаторами); назви ознак — через `get_feature_names_out()`;
2. **Permutation importance** на тестовій вибірці (функція `permutation_importance` з модуля `sklearn.inspection`, `scoring='f1_macro'`, не менше 10 повторень), застосована до всього пайплайну — тобто на *початкових* ознаках.

Дати відповіді на запитання:

- чи потрапили індикатори пропусків (`missingindicator_*`) до top-15 за MDI? Що це означає з огляду на аналіз механізму пропусків (п. 3.1)?
- наскільки узгоджуються MDI та permutation importance? Чому MDI може бути зміщеною для ознак із великою кількістю унікальних значень [10]?
- чому важливості однієї ознаки можуть суттєво відрізнятися між DT, RF і XGB?

3.8 Аналіз розбіжностей між моделями

Об'єкт вважається *об'єктом розбіжності*, якщо прогнози трьох фінальних моделей не всі однакові (достатньо, щоб хоча б одна модель відрізнялася).

Знайти не менше трьох таких об'єктів у тестовій вибірці та подати їх у таблиці (табл. 7) за зразком:

Зразок таблиці розбіжностей

Об'єкт	Справжній клас	Decision Tree	Random Forest	XGBoost
#17	3	2	3	3
#48	5	5	2	5
#93	2	4	2	2

Якщо в тестовій вибірці розбіжностей менше трьох (через ефект стелі це цілком можливо), доповнити їх об'єктами з тренувальної вибірки, використовуючи *out-of-fold* прогнози (`cross_val_predict`) — і явно зазначити це у звіті. Прогнози моделей на їхніх же тренувальних даних для цього не використовувати.

Для одного об'єкта розбіжності провести детальний аналіз:

1. вивести всі ознаки з початковими значеннями (до імпутації) і позначити, які з них були пропущені та якими значеннями їх заповнено;
2. за допомогою `shap.TreeExplainer` обчислити SHAP-значення [11, 12] для кожної моделі та навести п'ять ознак із найбільшим за модулем внеском у прогнозований клас (для багатокласових моделей масив SHAP-значень має форму (`n_samples, n_features, n_classes`));
3. порівняти ці ознаки між моделями та пояснити причини розбіжності: роль імпутованих значень та індикаторів пропусків, “жорсткі” пороги одного дерева проти усереднення в ансамблях, близькість об'єкта до межі між класами (порівняти ймовірності `predict_proba`).

Типові помилки

1. **Витік під час імпутації:** `fit_transform` на всьому датасеті до поділу або до крос-валідації. Імпутація має бути лише всередині `Pipeline`.
2. **Витік під час оптимізації:** передача в `Optuna` заздалегідь оброблених `X_train_proc` або використання тестової вибірки в цільовій функції.
3. **Оптимістична оцінка:** подання `study.best_value` як оцінки якості моделі.
4. **Рідкісні класи та кількість блоків:** якщо клас має менше спостережень, ніж блоків крос-валідації, `StratifiedKfold` видає попередження, а `XGBoost` може завершитися помилкою, якщо клас відсутній у тренувальному блоці. Саме тому внутрішня крос-валідація має 3 блоки. Перевірте мінімальну кількість об'єктів класу на кожному рівні.
5. **Неоднакові блоки** для різних моделей унеможливають парне порівняння.

6. Відсутність фіксованого SEED — результати неможливо відтворити.

4 Вимоги до звіту

Звіт подається у форматі Jupyter Notebook, який має **виконуватися без помилок від початку до кінця** (*Restart & Run All*). До ноутбука додати `requirements.txt`. Кожен блок результатів супроводжується коротким текстовим коментарем (Markdown-клітинка).

Таблиці:

1. крос-валідація, варіант A, типові параметри;
2. крос-валідація, варіант B, типові параметри;
3. вкладена крос-валідація, варіант B, після оптимізації, + `best_value`;
4. найкращі гіперпараметри, знайдені Optuna;
5. результати статистичного порівняння;
6. якість фінальних моделей на тестовій вибірці;
7. об'єкти розбіжностей між прогнозами моделей.

Графіки:

1. розподіл класів цільової змінної;
2. частка пропусків за ознаками;
3. історія оптимізації та важливість гіперпараметрів (для кожної моделі);
4. важливості ознак: MDI та permutation importance (для кожної моделі);
5. абсолютна та нормалізована матриці плутанини найкращої моделі;
6. SHAP-внески для проаналізованого об'єкта розбіжності.

Висновок

У висновку (8–12 речень) відповісти на такі питання, спираючись на власні числові результати:

1. Яка модель найкраща за масою F1 і наскільки стабільний цей результат? Чи є її перевага статистично значущою?
2. Як індикатори пропусків вплинули на якість? Чи узгоджується це з результатами аналізу механізму пропусків і з важливостями ознак?
3. Наскільки відрізняються `best_value` та оцінка вкладеної крос-валідації? Що це говорить про ризик оптимістичної оцінки?

4. Яка модель отримала найбільший приріст після оптимізації і чому?
5. Які класи найскладніші для класифікації і чому (кількість прикладів, схожість ознак)?
6. Що показав аналіз об'єктів розбіжностей?
7. Яку модель доцільно обрати для практичного використання з урахуванням якості, стабільності, інтерпретовності та обчислювальних витрат?

Примітка: наперед визначеної “правильної” моделі не існує. Якщо результати відрізняються від очікуваних або різниці статистично незначущі — це не помилка, а результат, який треба проаналізувати.

Контрольні питання

1. Чим випадковий ліс відрізняється від окремого дерева рішень з погляду розкладу похибки на зміщення та дисперсію? Яку роль відіграє декореляція дерев?
2. У чому принципова різниця між бустингом і беггінгом? Який із методів схильніший до перенавчання з ростом кількості дерев і чому?
3. Чому macro F1 і balanced accuracy інформативніші за accuracy в умовах дисбалансу? У чому різниця між macro- та weighted-усередненням F1?
4. Опишіть механізми MCAR, MAR і MNAR. За якого механізму індикатори пропусків найкорисніші? Чи можна за даними однозначно відрізнити MAR від MNAR?
5. У яких місцях пайплайну може виникнути витік даних під час імпутації та оптимізації гіперпараметрів і як цьому запобігти?
6. Чому `best_value` оптимізації є оптимістично зміщеною оцінкою? Як працює вкладена крос-валідація і яку модель ми насправді оцінюємо в її зовнішньому циклі?
7. Чому звичайний парний t -тест не підходить для порівняння результатів крос-валідації? Яку поправку вносить тест Надо–Бенжіо?
8. Чим MDI-важливість відрізняється від permutation importance? Як корельовані ознаки впливають на кожну з них?
9. Що таке SHAP-значення і яку властивість (адитивність) вони задовольняють? Чим локальне пояснення відрізняється від глобальної важливості?
10. Як алгоритм XGBoost обробляє пропущені значення нативно (sparsity-aware split finding)?
11. Які переваги має Optuna (TPE-семплер, прунінг) порівняно з GridSearchCV та RandomizedSearchCV?